

Grafuri

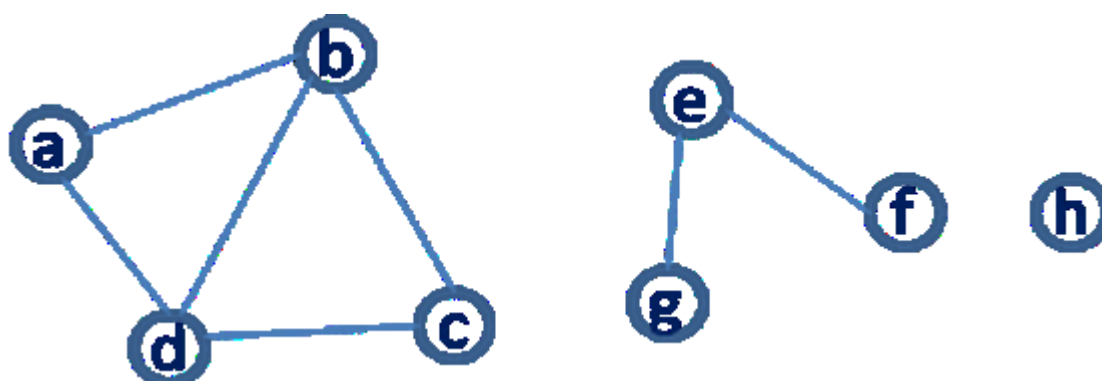
1 Considerente teoretice

În lucrarea de față vom prezenta cum se poate reprezenta un graf în Prolog și cum putem căuta drumul între două noduri.

1.1 Reprezentare

Un graf este reprezentat de două mulțimi: noduri și muchii. Dacă graful este orientat atunci vorbim despre: vârfuri și arce.

Exemplu de graf neorientat:



Există mai multe alternative de a reprezenta un graf în Prolog și le putem clasifica după următoarele criterii:

A. Tipul de reprezentare:

1. O colecție de noduri și o colecție de muchii
2. O colecție de noduri și lista asociată de vecini

B. Locul unde sunt salvate

In memoria principală, ca un data object (de ex. într-o structură)

In baza de cunoștințe, într-o colecție de predicate de tip axiomă/fapt

În consecință, patru reprezentări principale sunt posibile (alte abordări există, dar pentru scopul acestei lucrări, acestea sunt suficiente) :

A1B2: O colecție de predicate *node* și *edge*, stocate ca fapte (forma edge-clause)

Code

```
node(a).  
node(b).  
%etc
```

```
edge(a,b).
edge(b,a).
edge(b,c).
edge(c,b).
%etc
```

Avem nevoie de predicatele de tip *node* pentru cazurile în care avem noduri izolate. Ca o alternativă, nodurile izolate trebuie să fie specificate ca având un nod între ele și nil: `edge(f, nil)`. Dacă graful este unul neorientat, putem scrie predicatul în modul următor (pentru a evita nevoia de a scrie edge-urile în ambele direcții):

Code

```
is_edge(X,Y):- edge(X,Y); edge(Y,X).
```

A2B2: O colecție de noduri și listele asociate de vecini, stocate în faptele predicatului *neighbor/2* (forma neighbor list-clause)

Code

```
neighbor(a, [b, d]).
neighbor(b, [a, c, d]).
neighbor(c, [b, d]).
neighbor(h, []).
%etc
```

A1B1: O pereche formată dintr-o listă de noduri și o listă de muchii, stocat ca un fapt (forma graph-term)

Query

```
?- Graph = graph([a,b,c,d,e,f,g,h], [e(a,b), e(b,a), ... ]).
```

A2B1: O listă de perechi: nod, listă asociată de vecini, stocat ca un data object (forma neighbor list-list)

Query

```
?- Graph = [n(a, [b,d]), n(b, [a,c,d]), n(c, [b,d]), n(d, [a,b,c]), n(e, [f,g]), n(f, [e]),
n(g, [e]), n(h, [])].
```

Cea mai potrivită reprezentare depinde de problemă. Prin urmare, este convenient să știm modurile prin care putem să transformăm între diferite reprezentări de grafuri. Aici, furnizăm un exemplu de conversie din forma de *neighbor list-clause* (A2B2) în forma de *edge-clause* (A1B2):

Code

```
% declarăm predicatul dinamic pentru a putea folosi retract
```

```

:-dynamic neighbor/2.
% predicatul neighbor este considerat a fiind static deoarece este
% introdus în fișier, doar prin adăugarea declarației de dinamicitate
% ne este permis să folosim operația de retract asupra lui

% un exemplu de graf - prima componentă conexă a grafului
neighbor(a, [b, d]).
neighbor(b, [a, c, d]).
neighbor(c, [b, d]).
%etc.

neighb_to_edge:-
    %extrage un predicat neighbor
    retract(neighbor(Node,List)),!,
    % și apoi îl procesează
    process(Node,List),
    neighb_to_edge.
neighb_to_edge. % dacă nu mai sunt predicate neighbor/2, ne oprim

% procesarea presupune adăugarea de predicate edge și node
% pentru un predicat neighbor, prima dată adăugăm muchiile
% până când lista devine vidă iar abia apoi predicatele de tip node
process(Node, [H|T]):- assertz(gen_edge(Node, H)), process(Node, T).
process(Node, []):- assertz(gen_node(Node)).

```

Inițial graful este salvat în baza de date a predicatelor. Predicatul *neighb_to_edge* citește o clauză a predicatului *neighbor* la un moment dat și procesează informația din fiecare clauză separat.

Predicatul *process/2* traversează lista de vecini (al doilea argument) a nodului curent (primul argument) și adaugă (prin assert) un nou fapt la predicatul *gen_edge* pentru fiecare vecin nou al nodului curent. La condiția de oprire (a doua clauză), ne oprim când lista de vecini devine vidă și facem assert la un nou fapt de tip *gen_node*.

Predicatul *neighb_to_edge* se folosește de recursivitate și retract pentru a traversa secvențial clauzele predicatului *neighbor* și se oprește la un moment dat când niciun *neighbor* nu mai poate fi găsit. Fără folosirea operației de retract, această implementare va rezulta într-o buclă infinită a primului fapt al predicatului *neighbor*. În acest punct, *neighb_to_edge* ajunge la a doua clauză unde se oprește (predicatul neavând argumente, nici 'condiția de oprire' nu are – este nevoie de această clauză pentru a evalua predicatul ca true, altfel ultimul retract va da fail și va rezulta în false).

Pentru a pune întrebări acestui predicat, avem sintaxa următoare:

Query

```
?- neighb_to_edge.  
true;  
false.
```

După cum putem observa, este insuficient să verificăm rezultatul. Două predicate predefinite adiționale trebuie folosite pentru a putea verifica. Primul predicat este cel de *retractall/1*, acesta ne garantează că predicatele stocate sunt doar din această execuție prin ștergerea tuturor clauzelor unui predicat dat. Al doilea predicat este cel de *listing/1*, care ne permite să afișăm toate clauzele unui predicat dat. Fiecare din acestea necesită propriul format al argumentului. Întrebarea devine:

Query

```
?- retractall(gen_edge(_,_)), neighb_to_edge, listing(gen_edge/2).  
:- dynamic gen_edge/2.  
  
gen_edge(a, b).  
gen_edge(a, d).  
gen_edge(b, a).  
gen_edge(b, c).  
gen_edge(b, d).  
gen_edge(c, b).  
gen_edge(c, d).  
true;  
false.
```

Această abordare prezintă un dezavantaj. Datorită utilizării retragerii, predicatul vecin va fi eliminat în întregime din baza de predicat și nu va mai putea fi utilizat în aceeași instanță de execuție. Verificați următoarea întrebare:

Query

```
?- retractall(gen_edge(_,_)), neighb_to_edge, listing(gen_edge/2),  
listing(neighbor/2).
```

1.1.1 Implementări alternative (Failure driven loop vs recursion)

Predicatul anterior, *neighb_to_edge*, poate fi implementat într-un mod echivalent prin utilizarea unui failure-driven loop (laboratorul anterior, efecte laterale). Predicatul *process/2* va rămâne același.

Code

```
neighb_to_edge_v2:-  
    neighbor(Node,List), % accesăm faptul  
    process(Node,List),  
    fail.
```

```
neighb_to_edge_v2.
```

Parcurgerea predicatului nu se schimbă și nu mai necesită declararea predicatului *neighbor* ca fiind dinamic, deoarece nu este necesară folosirea retract-ului. Citește argumentele fiecărei clauze a predicatului *neighbor* și continuă cu procesarea fiecăruia. Prin folosirea fail-ului, backtracking-ul este activat și o nouă clauză a *neighbor* este citit până când nu rămâne niciunul necitit. În acest punct, se ajunge la a doua clauză a *neighb_to_edge* și se oprește.

Prima implementare (folosind recursivitatea) distruge baza predicatului *neighbor/2* din cauza *retract*-ului. Astfel, a doua implementare este de preferat. Cu toate acestea, dacă dorim să folosim recursivitatea fără *retract*, se poate utiliza o a treia implementare, dar aceasta necesită un predicat *seen* pentru a verifica ce fapte ale *neighbor* au fost parcurse pentru a nu intra într-o buclă infinită.

Code

```
:-dynamic seen/1.

neighb_to_edge_v3:-
    neighbor(Node,List),
    not(seen(Node)),!,
    assert(seen(Node)),
    process(Node,List),
    neighb_to_edge_v3.
neighb_to_edge_v3.
```

1.2 Drumuri în graf

Știind reprezentările grafurilor, vom trece la traversarea lor. Vom începe cu un drum simplu între două noduri și vom ajunge la drumuri restricționate, drumuri optime și, în final, la ciclul Hamiltonian al unui graf.

1.2.1 Drum Simplu

Presupunem că graful este reprezentat prin predicate *node* și *edge*. Următorul predicat caută un drum între două noduri și returnează lista de noduri parcurse.

Code

```
% path(Source, Target, Path)
% drumul parțial începe cu nodul sursă - acesta este un wrapper
path(X, Y, Path):-path(X, Y, [X], Path).

% când sursa (primul argument) este egal cu destinația (al doilea
% argument), atunci știm că drumul a ajuns la final
% și putem unifica drumul parțial cu cel final
path(Y, Y, PPath, PPath).
path(X, Y, PPath, FPath):-
```

<code>edge(X, Z),</code>	<code>% căutăm o muchie</code>
<code>not(member(Z, PPath)),</code>	<code>% care nu a mai fost parcursă</code>
<code>path(Z, Y, [Z PPath], FPath).</code>	<code>% și o adăugăm în rezultatul parțial</code>

Urmărește execuția la:

Query

?- trace, path(a,c,R).

Puteți folosi exemplul de graf sau chiar să adăugați niște muchii la acesta. Ce se întâmplă când repetăm întrebarea?

1.2.2 Drum Restricționat

Drumul restricționat presupune trecerea printr-un anumit set de noduri în ordinea dată. Deoarece *path* returnează drumul în ordine inversă atunci vom aplica *reverse*.

Code

```
% restricted_path(Source, Target, RestrictionsList, Path)
restricted_path(X,Y,LR,P):-
    path(X,Y,P),
    reverse(P,PR),
    check_restrictions(LR, PR).

% verificăm dacă se respectă restricția
check_restrictions([],_):-!.
check_restrictions([H|TR], [H|TP]):-!, check_restrictions(TR,TP).
check_restrictions(LR, [_|TP]):-check_restrictions(LR,TP).
```

Predicatul *restricted_path/4* caută un drum între nodul sursă și destinație și verifică dacă acest drum satisface restricțiile specificate în LR (i.e. dacă trece printr-o secvență de noduri, specificată în LR), folosind predicatul *check_restrictions/2*, care face de fapt verificarea.

Predicatul *check_restrictions/2* traversează lista de restricții (primul argument) și lista care reprezintă drumul (al doilea argument) simultan, cât timp primele elemente din cele două liste coincid (clauza 2). La momentul în care sunt diferite, vom continua doar cu a doua listă (clauza 3). Predicatul reușește când prima listă devine vidă (clauza 1).

Întrebare: Ce se întâmplă dacă mutăm condiția de oprire ca ultima clauză? Avem nevoie de tăierea de backtracking în condiția de oprire?

Urmărește execuția la:

Query

?- trace, check_restrictions([2,3], [1,2,3,4]).

```
?- trace, check_restrictions([1,3], [1,2,3,4]).
?- trace, check_restrictions([1,3], [1,2]).
?- trace, restricted_path(a, c, [a,c,d], R).
```

1.2.3 Drum Optim

Considerăm un drum optim între două noduri într-un graf ca acel drum care are lungimea minimă. O abordare de a găsi drumul optim este de a genera toate drumurile prin backtracking și selecția drumului optim. Evident, această abordare este inefficientă. În loc să facem asta, în timpul procesului de backtracking, vom reține soluția optimă parțială folosind efecte laterale (în baza de predicate) și o vom actualiza când o soluție mai bună este găsită:

Code

```
:- dynamic sol_part/2.

% optimal_path(Source, Target, Path)
optimal_path(X,Y,Path):-
    asserta(sol_part([], 100)),    % 100 = distanța maximă inițială
    path(X, Y, [X], Path, 1).
optimal_path(_,_,Path):-
    retract(sol_part(Path,_)).

% path(Source, Target, PartialPath, FinalPath, PathLength)
% când ținta este egală cu sursa, salvăm soluția curentă
path(Y,Y,Path,Path,LPath):-
    % scoatem ultima soluție
    retract(sol_part(_,_)),!,
    % salvăm soluția curentă
    asserta(sol_part(Path,LPath)),
    % căutăm o altă soluție
    fail.
path(X,Y,PPath,FPath,LPath):-
    edge(X,Z),
    not(member(Z,PPath)),
    % calculăm distanța parțială
    LPath1 is LPath+1,
    % extragem distanța de la soluția precedentă
    sol_part(_,Lopt),
    % dacă distanța curentă nu depășește distanța precedentă
    LPath1<Lopt,
    % mergem mai departe
    path(Z,Y,[Z|PPath],FPath,LPath1).
```

Predicatul *path/4* generează, prin backtracking, toate drumurile care sunt mai bune decât soluția curentă parțială și actualizează soluția curentă parțială când un drum mai scurt este găsit. Când o soluție mai bună decât cea curentă este găsită, predicatul înlocuiește soluția optimă veche în baza de predicate (clauza 1) și după continuă căutarea, prin pornirea mecanismului de backtracking (folosind fail).

Urmărește execuția la:

Query

```
?- trace, optimal_path(a,c,R).
```

Rețineți. Când folosim *assert/retract*, trebuie să ne asigurăm că “curățăm”, trebuie să verificăm că nicio clauză nedorită nu rămâne stocată în baza de predicate după execuția întrebărilor (nu sunt afectate de backtracking!).

1.2.4 Ciclu Hamiltonian

Un ciclu Hamiltonian este un ciclu care trece prin toate nodurile o singură dată (cu excepția nodului de start care este începutul și sfârșitul acestui ciclu). Evident, nu toate grafurile conțin un astfel de ciclu. Predicatul *hamilton/3* este prezentat mai jos:

Code

```
% hamilton(NumOfNodes, Source, Path)
hamilton(NN, X, Path):- NN1 is NN-1, hamilton_path(NN1, X, X, [X],Path).
```

Predicatul *hamilton_path/5* vă rămâne de implementat. Predicatul acesta ar trebui să caute un drum închis începând din X, de lungime NN1 (numărul de noduri din graf, minus 1).

Urmăriți execuția predicatului *hamilton/3* folosind graful dat ca exemplu.

1.3 O implementare mai bună a drumurilor

Drumurile sunt salvate în ordine inversă, o posibilă îmbunătățire ar fi salvarea lor în ordinea corectă. Deoarece adăugăm la începutul celui de-al treilea argument, care funcționează într-o abordare forward, rezultatele sunt inversate. Totuși, putem adăuga direct la ieșire, păstrând al treilea argument doar ca o listă a nodurilor vizitate.

Code

```
path1(X, Y, Path):- path1(X, Y, [X], Path).

path1(Y, Y, _, []).
path1(X, Y, PPath, [Z|FPath]):-
    edge(X, Z),
    not(member(Z, PPath)),
    path1(Z, Y, [Z|PPath], FPath).
```

Am ajuns la o implementare mai bună, care nu necesită inversarea drumului.

Totuși, aceasta nu este încă cea mai bună implementare posibilă. De ce? Predicatul *edge/2* reprezintă un singur graf, așadar putem rula această căutare doar pe un singur graf; dacă dorim să o folosim pe un alt graf, trebuie să modificăm implementarea. O soluție este utilizarea predicatului *call/n*, discutat în ultima sesiune de laborator.

Acest predicat cu *call/n* datorită $[Z|PPath]$ ar putea să crească nelimitat, motiv pentru care SWISH-Prolog nu va permite evaluarea și va da timeout. O soluție este construirea acumulatorului în $PPath1=[Z|PPath]$ și apelarea recursivă cu acesta sau utilizarea unui *meta_predicate* ca și mai jos.

Code

```
% meta_predicate indica ca primul argument al path1/4
% este un predicat in sine cu 2 argumente
:- meta_predicate path2(2, ?, ?, ?).

path2(Graph, X, Y, [X|Path]):- path2(Graph, X, Y, [X], Path).

path2(_, Y, Y, _, []).
path2(G, X, Y, PPath, [Z|FPath]):-
    call(G, X, Z),                % apelăm G(X, Z)
    not(member(Z, PPath)),
    path1(G, Z, Y, [Z|PPath], FPath).
```

Urmărește execuția la:

Query

```
?- trace, path2(edge, a, c, P).
```

2 Exerciții

Fiecare exercițiu are alocat propriul său graf. Așadar, în implementarea ta ar trebui să folosești muchiile acelui graf. De exemplu, când rezolvi Exercițiul 1, predicatul ar trebui să apeleze *edge_ex1/2*.

1. Rescrie *optimal_path/3* astfel încât să funcționeze pe grafuri ponderate: atașai o pondere pentru fiecare muchie din graf și calculeți drumul de cost minim dintr-un nod sursă la un nod destinație.

Predicatul *edge* va avea 3 parametrii.

Code

```
edge_ex1(a,c,7).  
edge_ex1(a,b,10).  
edge_ex1(c,d,3).  
edge_ex1(b,e,1).  
edge_ex1(d,e,2).
```

Query

```
?- optimal_weighted_path(a, e, P).  
P = [e, b, a]
```

2. Continuă implementarea la ciclul Hamiltonian folosind predicatul *hamilton/3*.

Code

```
edge_ex2(a,b).  
edge_ex2(b,c).  
edge_ex2(a,c).  
edge_ex2(c,d).  
edge_ex2(b,d).  
edge_ex2(d,e).  
edge_ex2(e,a).
```

Query

```
?- hamilton(5, a, P).  
P = [a, e, d, c, b, a]
```

3. Scrieți predicatul *euler/3* care poate să găsească drumuri Euleriane într-un graf dat de la un nod dat.

Code

```
% euler(NE, S, R). - unde S este nodul sursă  
% și NE este numărul de muchii din graf  
edge_ex3(a,b).  
edge_ex3(b,e).  
edge_ex3(c,a).  
edge_ex3(d,c).  
edge_ex3(e,d).
```

Query

```
?- euler(5, a, R).  
R = [[c, a], [d, c], [e, d], [b, e], [a, b]]
```

4. Scrie predicatul *cycle(X,R)* care găsește un ciclu ce pornește din nodul *X* dintr-un graf *G* (folosind reprezentarea *edge/2*) și pune rezultatul în *R*. Predicatul trebuie să returneze toate ciclurile prin backtracking. În plus, predicatul trebuie să primească predicatul *edge* care definește graful ca și input (*sugestie*: predicatul *call/n*).

Code

```
edge_ex4(a,b).  
edge_ex4(a,c).  
edge_ex4(c,e).  
edge_ex4(e,a).  
edge_ex4(b,d).  
edge_ex4(d,a).
```

Query

```
?- cycle(edge_ex4, a, R).  
R = [a,d,b,a] ;  
R = [a,e,c,a] ;  
false
```

5. Scrieți predicatul $cycle(X,R)$ din exercițiul anterior folosind reprezentarea $neighbour/2$. În plus, predicatul trebuie să primească predicatul $neighb$ care definește graful ca și input (sugestie: predicatul $call/n$).

Code

```
neighb_ex5(a, [b,c]).  
neighb_ex5(b, [d]).  
neighb_ex5(c, [e]).  
neighb_ex5(d, [a]).  
neighb_ex5(e, [a]).
```

Query

```
?- cycle_neighb(neighb_ex5, a, R).  
R = [a,d,b,a] ;  
R = [a,e,c,a] ;  
false
```

6. Scrieți un predicat care convertește din A1B2 (edge-clause) în A2B2 (neighbor list-clause).

Code

```
edge_ex6(a,b).  
edge_ex6(a,c).  
edge_ex6(b,d).
```

Query

```
?- retractall(gen_neighb(_,_)), edge_to_neighb, listing(gen_neighb/2).  
:- dynamic gen_neighb_list/2.  
  
gen_neighb(c, []).  
gen_neighb(d, []).  
gen_neighb(a, [c, b]).  
gen_neighb(b, [d]).  
true
```

7. Predicatul *restricted_path/4* găsește un drum între nodul sursă și cel destinație și verifică dacă drumul găsit conține nodurile din lista de restricții. Acest predicat folosește recursivitate înainte, ordinea nodurilor trebuie inversată în ambele liste – lista de drum și de restricții. Motivați de ce această strategie nu este eficientă (urmăriți ce se întâmplă). Scrieți un predicat mai eficient care caută un drum restricționat între nodul sursă și cel destinație.

Code

```
edge_ex6(a,b).  
edge_ex6(b,c).  
edge_ex6(a,c).  
edge_ex6(c,d).  
edge_ex6(b,d).  
edge_ex6(d,e).  
edge_ex6(e,a).
```

Query

```
? - restricted_path_efficient(a, e, [c,d], P2).  
P = [a, b, c, d, e];  
P = [a, c, d, e];  
false
```

8. Scrie o serie de predicate care să rezolve problema Lupul-Capra-Varza: “un fermier trebuie să mute în siguranță, de pe malul nordic pe malul sudic, un lup, o capră și o varză. În barcă încap maxim doi și unul dintre ei trebuie să fie fermierul. Dacă fermierul nu este pe același mal cu lupul și capra, lupul va mânca capra. Același lucru se întâmplă și cu varza și capra, capra va mânca varza.”

Sugestii:

- Puteți alege să encodați spațiul de stări ca instanțe a unei configurări ale celor 4 obiecte (Fermier, Lup, Capră, Varză): reprezentate ca o listă cu poziția celor 4 obiecte [Fermier, Lup, Capră, Varză] sau ca o structură complexă (ex. F-W-G-C, sau state(F,W,G,C)).
- starea inițială este [n,n,n,n] (toți sunt în nord) și starea finală este [s,s,s,s] (toți au ajuns în sud); pentru reprezentarea folosind liste a stărilor (ex. dacă Fermierul trece lupul -> [s,s,n,n], această stare nu ar trebui să fie validă deoarece capra mănâncă varza).
- la fiecare trecere Fermierul își schimbă poziția (din „n” în „s” sau invers) și **cel mult** încă un participant (Lupul, Capra, Varza).
- problema poate fi văzută ca o problema de căutare a drumului într-un graf.